MediCare — Domain Class Diagram

Conceptual domain model derived from the 11-table baseline. Foreign keys are expressed as associations. {P} = proposed attribute not stated in the baseline documents.

User / Patient / Doctor — shared account data lives in User; Patient and Doctor are separate profile classes (UserId unique in each), mirroring the baseline schema, so they are associations rather than subclasses. With ASP.NET Identity the physical tables are AspNetUsers and role tables (framework refinement). Passwords are never stored in plain text.

DoctorSchedule

- ScheduleId: int
- DayOfWeek: DayOfWeek
- StartTime: TimeOnly
- EndTime: TimeOnly
- SlotLength: int (minutes)

+ ValidateTimeRange(): bool
+ UpdateSchedule()

0..* works to 1

User

- UserId: int
- FullName: string
- Email: string {unique}
- PhoneNumber: string
- PasswordHash: string (identity-managed)
- IsActive: bool
- Role: UserRole

+ UpdateProfile()
+ ActivateAccount()
+ DeactivateAccount()

1 has doctor profile 1

1 receives 0..*

Notification

- NotificationId: int
- Type: string
- Message: string
- Channel: NotificationChannel
- Status: string {P}
- CreatedAt: DateTime {P}
- SentAt: DateTime {P}

+ CreateNotification()
+ MarkAsSent()
+ MarkAsFailed()

Delivery is done by application services via SignalR, Twilio (SMS) and SendGrid (e-mail); they are not domain classes or tables. Only the persisted notification is modeled.

Specialization

- SpecializationId: int
- Name: string {unique}
- Description: string {P}

+ Create()
+ Update()
+ Delete()

1 classified by 0..*

Doctor

- DoctorId: int
- LicenseNumber: string
- ConsultationFee: decimal
- ProfileImage: string
- ApprovalStatus: DoctorApprovalStatus

+ UpdateProfile()
+ ConfigureWorkingHours()
+ ManageVacations()
+ ConfirmAppointment()
+ RejectAppointment(reason)
+ CreateMedicalRecord()
+ IssuePrescription()
+ MarkAppointmentNoShow()

0..1 1

1 has patient profile 0..1

Patient

- PatientId: int
- DateOfBirth: Date
- Gender: string
- MedicalHistory: string
- Allergies: string

+ UpdatePersonalInformation()
+ UpdateMedicalHistory()
+ UpdateAllergies()
+ ViewAppointments()
+ ViewMedicalRecords()
+ ViewPrescriptions()

enumeration
AppointmentStatus

Pending
Confirmed
Rejected
Cancelled
Completed
NoShow

MarkAppointmentNoShow(): the no-show policy is not defined in the documents — to be confirmed.

1 takes 0..*

1 attends 0..*

Appointment

- AppointmentId: int
- StartDateTime: DateTime
- EndDateTime: DateTime {P}
- Status: AppointmentStatus
- AppointmentType: AppointmentType {optional}
- Reason: string
- Notes: string
- RejectionReason: string {P}
- CreatedAt: DateTime {P}
- RowVersion: byte[] (concurrency token)

+ Book()
+ Confirm()
+ Reject(reason)
+ Cancel()
+ Reschedule(newStart)
+ Complete()
+ MarkNoShow()
+ IsSlotAvailable(): bool
+ ValidateCancellationPolicy(): bool

0..* books 1

enumeration
AppointmentType

NewVisit
FollowUp
CheckUp

Legend: '-' private attribute, '+' public operation. {P} proposed attribute (not in the baseline documents). {optional} AppointmentType is a 'Should' feature. Foreign keys (PatientId, DoctorId, UserId, SpecializationId, MedicalRecordId, PrescriptionId, AppointmentId) are shown as associations. Business rules beyond the box are in notes; application services and controllers are intentionally not part of this domain view. MedicalDocuments is an optional extension and is not shown.

DoctorVacation

- VacationId: int
- StartDate: Date
- EndDate: Date
- Reason: string {P}

+ ValidateAgainstConfirmedAppointments(): bool
+ AddVacation()

A doctor cannot add vacation days over appointments that are already Confirmed.

Appointment rules: (1) no two active appointments of the same Doctor may overlap in time — enforced by the service and a concurrency check (RowVersion) at save; (2) Cancel() / Reschedule() only if more than 24 h remain; (3) a Pending appointment with no doctor response after 24 h is cancelled automatically by a background service.

enumeration
DoctorApprovalStatus

Pending
Approved
Rejected

1 documented by 0..1
{unique AppointmentId}

MedicalRecord

- MedicalRecordId: int
- Symptoms: string
- Diagnosis: string
- TreatmentNotes: string
- CreatedAt: DateTime {P}

+ CreateRecord()
+ UpdatePermittedDetails()

1 versions 0..*
{unique (record, version)}

Prescription

- PrescriptionId: int
- VersionNumber: int
- IssuedAt: DateTime {P}

+ Issue()
+ CreateNewVersion()
+ Print()

enumeration
NotificationChannel

InApp
SMS
Email

1 contains 1..*

PrescriptionItem

- PrescriptionItemId: int
- MedicationName: string
- Dose: string
- Frequency: string
- Duration: string
- Instructions: string

+ ValidateMedicationDetails(): bool

enumeration
UserRole

Patient
Doctor
Admin

One record per appointment (unique AppointmentId); created only by the assigned doctor for Confirmed (or Completed) appointments. The patient is reached through the Appointment, so there is no direct Patient–MedicalRecord link.

Dashed = optional ('Should' priority: appointment types)

Issued prescriptions are immutable: a change creates a new version (VersionNumber + 1); unique (MedicalRecordId, VersionNumber). Composition assumes items are created and removed with their prescription version.